Database Systems: Ordering Joins in a Query Optimiser

DAA Assignment 2026

Batch 3

Asraf M – 24I307

Mazin J – 24I330

Srikesav M – 24I360

Stalin K- 24I363

Tarun Kumar KPS – 24I368

Dynamic Programming approach

PSG College of Technology

ALGORITHM DESIGN

Problem Overview

- A query may join multiple database relations.
- Different join orders can produce different costs.
- The goal is to find the minimum-cost join order.
- Efficient ordering improves query execution.

Problem Scenario

- Consider five relations: R1, R2, R3, R4 and R5.
- The relations are joined in a left-to-right chain.
- Each relation is represented using its dimensions.
- We compare Dynamic Programming with a Greedy approach.

Input Dimensions

- Dimension array: p = [40, 20, 30, 10, 30, 15].
- Five relations require six dimension values.
- A join plan determines the order of combining relations.
- The same input is used to compare both approaches.

Join Cost Model

- A join combines two adjacent relation subchains.
- Cost = $p[i-1] \times p[k] \times p[j]$.
- The split point k separates the left and right subchains.
- The objective is to minimise the total join cost.

Possible Join Orders

- Five relations have 14 possible parenthesisations.
- The number of possibilities grows rapidly with relation count.
- For 15 relations, there are 2,674,440 parenthesisations.
- Checking every possible order becomes expensive.

Chosen Design Technique

- Dynamic Programming solves the problem systematically.
- It uses optimal substructure and overlapping subproblems.
- Subchain results are stored and reused.
- The technique is based on Matrix Chain Multiplication.

DP State and Base Case

- m[i][j] stores the minimum cost for joining Ri through Rj.
- s[i][j] stores the split point that gives the minimum cost.
- A single relation needs no join operation.
- Therefore, m[i][i] = 0.

DP Recurrence

- Try every split k between i and j.
- Combine the optimal left and right subchain costs.
- Add the cost of joining the two resulting subchains.
- Store the minimum over all possible split points.

DP Algorithm

- Initialise the cost table with zero diagonal entries.
- Evaluate subchains in increasing order of length.
- For each subchain, examine every valid split point.
- Record the minimum cost and its corresponding split.

Pseudocode

- For each chain length from 2 to n:
- For each valid starting index i, calculate j.
- For every k from i to j−1, calculate the total cost.
- Update m[i][j] and s[i][j] when a lower cost is found.

Worked Example: Pair Costs

- R1 JOIN R2: 40 × 20 × 30 = 24,000.
- R2 JOIN R3: 20 × 30 × 10 = 6,000.
- R3 JOIN R4: 30 × 10 × 30 = 9,000.
- R4 JOIN R5: 10 × 30 × 15 = 4,500.

DP Trace: Optimal Subchains

- The minimum cost for R2 JOIN R3 is 6,000.
- The minimum cost for R4 JOIN R5 is 4,500.
- The minimum cost for R1 through R3 is 14,000.
- The minimum cost for R3 through R5 is 9,000.

Optimal Join Order

- First join R2 with R3.
- Next join R1 with the result of R2 JOIN R3.
- Separately join R4 with R5.
- Finally join the two resulting subchains.

Optimal Cost Calculation

- R2 JOIN R3 costs 6,000.
- R1 JOIN (R2 JOIN R3) costs 8,000 more.
- R4 JOIN R5 costs 4,500.
- The final join costs 6,000; total = 24,500.

Correctness: Base and Optimality

- A single relation has zero join cost.
- Every possible split is considered by the recurrence.
- Each side uses its minimum-cost subchain solution.
- A non-optimal subchain could be replaced to reduce total cost.

Correctness: Inductive Argument

- The base case is correct for subchains of length one.
- Assume all shorter subchains have optimal costs.
- The recurrence combines optimal solutions for every split.
- Thus, the computed cost is optimal for the current subchain.

Greedy Alternative

- Greedy repeatedly selects the cheapest adjacent pair.
- The initial cheapest pair is R4 JOIN R5 at 4,500.
- It makes a locally cheapest choice at each step.
- A locally optimal choice need not give a globally optimal plan.

Greedy Result and Comparison

- Greedy total join cost = 30,000.
- Dynamic Programming total join cost = 24,500.
- Greedy costs 5,500 more than Dynamic Programming.
- The Greedy result is approximately 22.45% higher.

Complexity Analysis

- There are $O(n^2)$ subchains in the DP table.
- Each subchain considers up to O(n) split points.
- Time complexity = $O(n^3)$.
- Space complexity = $O(n^2)$.

Worst-Case and Scalability

- Equal dimensions can create many equally optimal join orders.
- Ties do not increase the number of DP split evaluations.
- The DP algorithm still performs $O(n^3)$ work for a chain.
- Larger join graphs may require more advanced optimisation.

Empirical Analysis

- Generate test inputs for 5, 10, 15, 20 and 25 relations.
- Measure both algorithms over repeated executions.
- Use median execution time to reduce timing noise.
- Store measured costs and times in results/timing.csv.

Conclusion and Real-World Relevance

- Join ordering has a direct impact on query execution cost.
- Dynamic Programming finds the optimal order for the chain model.
- The example reduces cost from 30,000 to 24,500.
- Database optimisers use join-order search strategies to improve queries.